



CampusRide

One Request. Instant Ride. Real-Time Campus Mobility.

IIT Roorkee Hackathon · Full-Stack Web Application

Node.js

React 18

MongoDB

Socket.IO

JWT

Tailwind CSS

Problem Statement

The IIT Roorkee campus spans a large area and relies on e-rickshaws for last-mile transport. Currently, ride coordination is fragmented — passengers wait without visibility and drivers operate without a centralised demand system.



No Visibility

Passengers cannot see available drivers or estimate wait times



Slow Coordination

Ride matching happens via phone calls and informal word-of-mouth



Zero Analytics

No data on demand patterns, peak hours, or popular locations



No Accountability

No ride history, ratings, or feedback mechanism for drivers

Our Solution

CampusRide — A centralised digital platform connecting passengers and drivers in real time



Instant Ride Requests

Passengers request rides with pickup & destination in seconds



Real-Time Notifications

Drivers get instant WebSocket alerts for new ride requests



Full Ride Lifecycle

Requested → Accepted → In Progress → Completed / Cancelled



Atomic Assignment

MongoDB ensures only one driver can claim each ride



Driver Dashboard

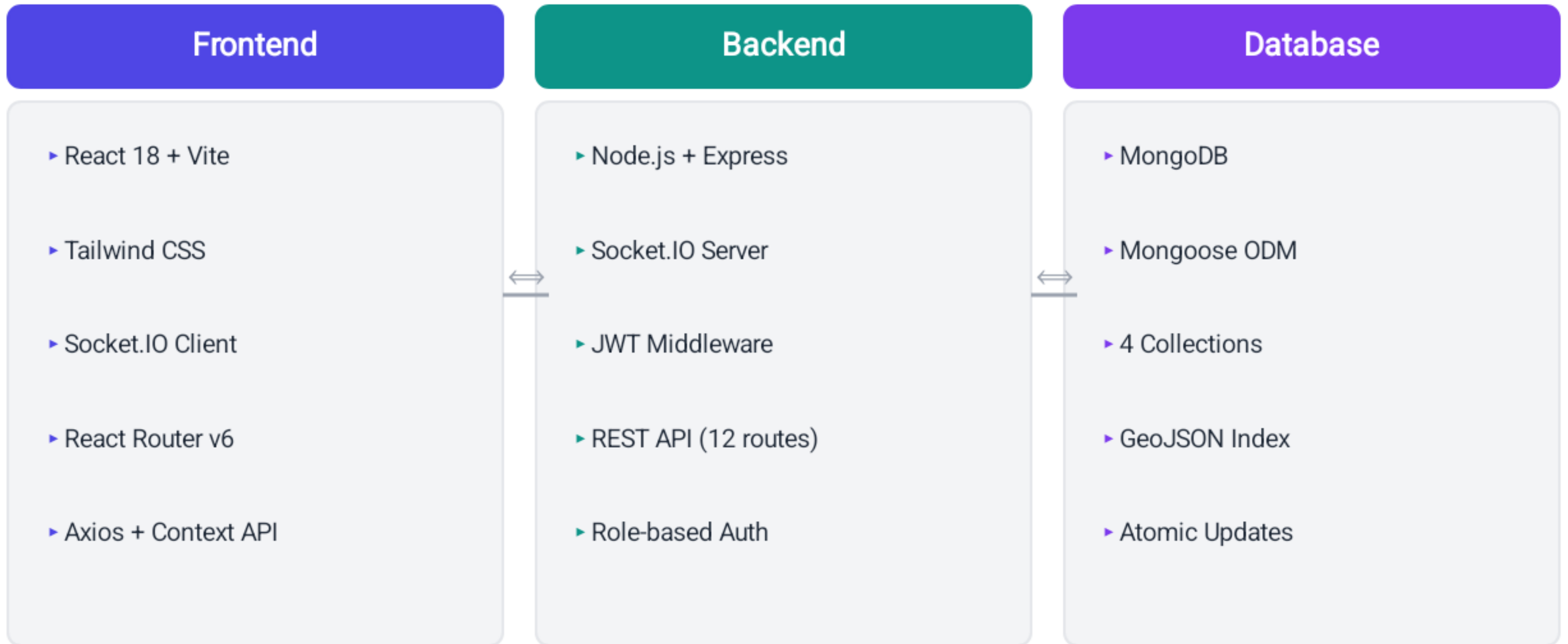
Live stats, 7-day chart, ratings and ride history



Ratings & Feedback

Passengers rate rides; drivers build reputation scores

System Architecture



⚡ Real-Time Layer: Socket.IO WebSockets — ride events broadcast to targeted rooms (passenger / drivers_room)

Real-Time Communication

Powered by Socket.IO WebSockets

ride:new_request

Server → Drivers room

New ride broadcast to all online drivers instantly

ride:accepted

Server → Passenger

Passenger notified the moment a driver accepts

ride:taken

Server → Drivers room

Ride removed from all other drivers' queues

ride:status_update

Server → All

Live status changes pushed to every connected client

driver:availability_changed

Server → All

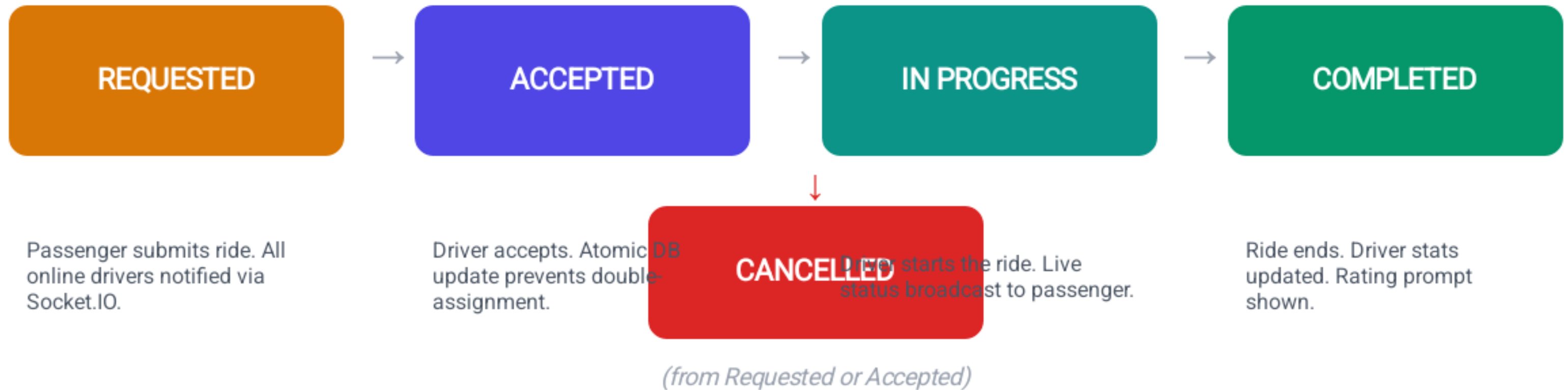
Driver online/offline state synced in real time

driver:location_update

Client → Server

Driver GPS ping stored and relayed to passenger

Ride Lifecycle & State Machine



 **Atomic Acceptance:** MongoDB findOneAndUpdate with status:'requested' filter ensures exactly one driver can accept — even under concurrent requests. A 409 Conflict is returned to all others.

Database Schema — 4 Collections

USER

_id (PK)

name

email

password (bcrypt)

phone

role: passenger|driver

DRIVER

_id (PK)

user (FK → User)

vehicleNumber

isOnline

averageRating

socketId

RIDE

_id (PK)

passenger (FK → User)

driver (FK → Driver)

pickup / destination

status (enum)

timestamps

RATING

_id (PK)

ride (FK → Ride)

passenger + driver (FK)

score (1-5)

feedback (text)

User 1 — 1 Driver · User 1 — ∞ Ride · Driver 1 — ∞ Ride · Ride 1 — 0..1 Rating

Driver Dashboard & Analytics



247

Total Rides



12

Today's Rides



4.8



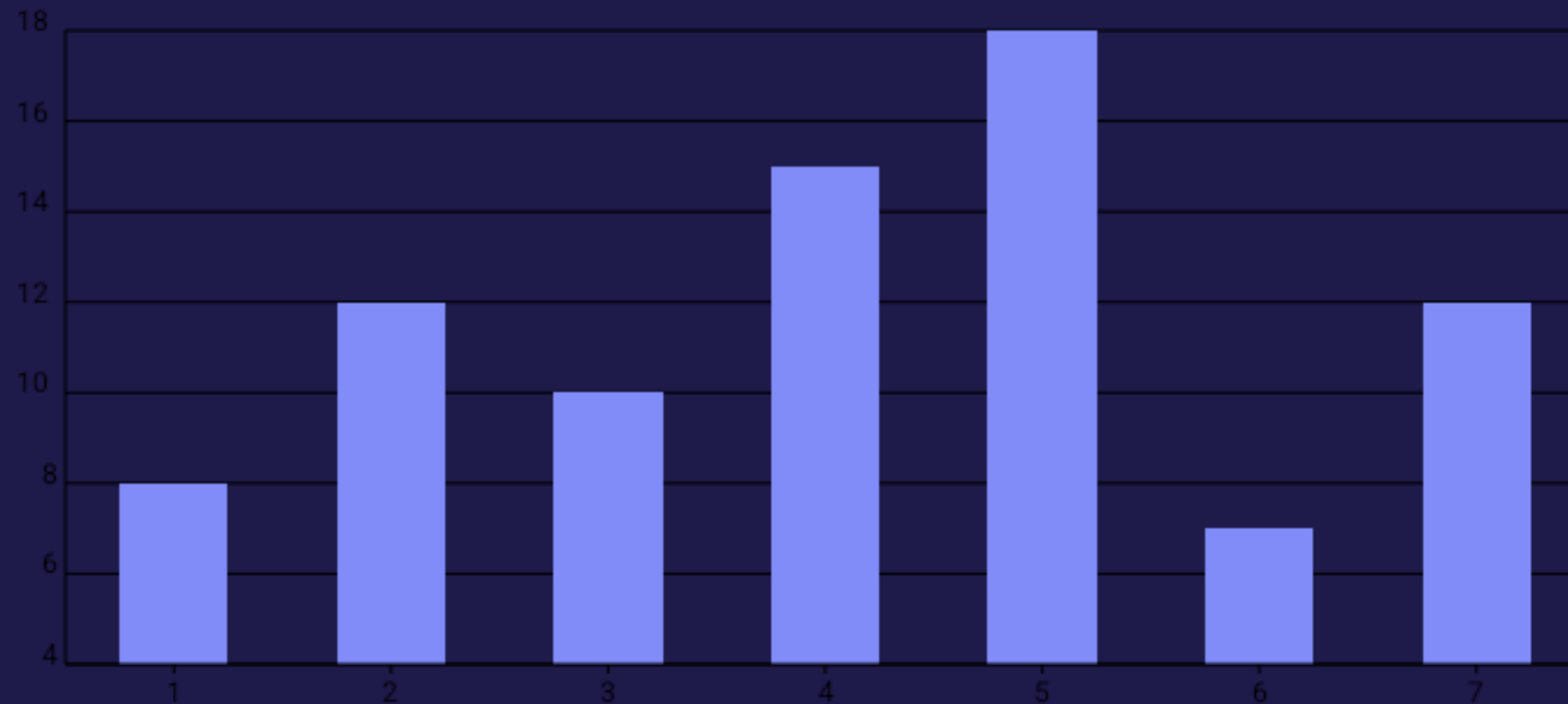
Avg Rating



198

Total Ratings

Rides — Last 7 Days



Dashboard Features

- ✓ Live incoming ride notifications
- ✓ One-click Accept / Ignore rides
- ✓ Start Ride & Complete Ride buttons
- ✓ 7-day bar chart (Recharts)
- ✓ Recent rides activity table
- ✓ Average rating display
- ✓ Online / Offline toggle

Technology Stack

Layer	Technology	Why We Chose It
Frontend	React 18 + Vite	Fast HMR in dev, optimised bundles in prod, component model suits real-time UI
Styling	Tailwind CSS	Utility-first CSS enables rapid UI prototyping without leaving JSX
Backend	Node.js + Express.js	Non-blocking I/O handles concurrent WebSocket connections efficiently
Real-Time	Socket.IO	Abstracts WebSockets with rooms, namespaces, and automatic fallback
Database	MongoDB + Mongoose	Flexible document model suits evolving schemas; GeoJSON for location data
Auth	JWT (JSON Web Tokens)	Stateless — works identically for REST API and Socket.IO handshake auth
Charts	Recharts	React-native chart library; responsive bar charts for driver analytics
HTTP Client	Axios	Interceptors auto-inject JWT token and handle 401 global redirects

Key Engineering Highlights



Atomic Ride Assignment

`findOneAndUpdate` with `{status:'requested'}` filter — only one driver wins. 409 Conflict returned to all others. No distributed locks needed.



Socket.IO Room Architecture

Each user joins room `user_{id}`. All drivers share `drivers_room`. Targeted delivery without broadcasting to unrelated clients.



JWT in Socket.IO Handshake

Socket.IO auth middleware validates JWT from `handshake.auth.token` — same security model as the REST API, no separate session store.



Role-Based Access Control

`protect()` + `requireRole()` middleware chain guards every route. Passengers and drivers see only their own data and actions.



Thank You!

CampusRide — One Request. Instant Ride. Real-Time Campus Mobility.

- ✓ Full-stack real-time ride management platform
- ✓ Socket.IO WebSockets for instant updates
- ✓ Atomic ride assignment — no double-booking
- ✓ JWT auth across REST API and WebSockets
- ✓ Driver dashboard with analytics & charts

Built with Node.js · React 18 · MongoDB · Socket.IO · Tailwind CSS · JWT